

LABORATORIO 3

MQTT hacia Azure IoT Central Protocolo visible, mediciones y comparación con el SDK

1. Objetivos

- Explicar qué protocolo usa realmente un dispositivo cuando habla con IoT Central (MQTT sobre TLS, DPS y luego el hub).
- Publicar las mismas ≥ 3 variables del Lab 2 hacia IoT Central por dos caminos: SDK Python y cliente MQTT explícito (en la VM).
- Mantener el ESP32 de Wokwi enviando por MQTT (firmware Arduino) a la misma aplicación.
- Medir tamaño de payload, intervalo, tiempo hasta ver el dato en IoT Central y comportamiento ante corte breve.
- Documentar qué QoS acepta IoT Central y qué implica eso frente a QoS 0 / 1 / 2 de la especificación MQTT.

2. Protocolo de este laboratorio

El protocolo de trabajo es **MQTT**, pero no contra un broker de laboratorio suelto: el destino es **Azure IoT Central**. Por debajo, Central usa Device Provisioning Service y un IoT Hub. El dispositivo se conecta a **mqtt://** (puerto 8883) o MQTT sobre WebSockets (443) con autenticación SAS. La telemetría se publica en el topic de eventos del dispositivo. IoT Hub/Central acepta QoS 0 y QoS 1; **no soporta QoS 2**.

Camino	Dónde	Qué se observa
SDK azure-iot-device	Python en la VM (Lab 2)	El SDK oculta topics y token. Sirve de baseline.
MQTT explícito (paho u otro)	Misma VM	Username, SAS, topic de telemetría y QoS a la vista.
Arduino / ESP32	Wokwi	El firmware ya habla MQTT/TLS con Central. Se registran logs de publish.

No se pide levantar Mosquitto/EMQX como destino principal. Un broker local solo se usa, si acaso, como contraste de una línea en la tabla (opcional).

3. Prerrequisitos

- Laboratorio 2 entregado: aplicación IoT Central, plantilla, flota, Python en VM y ESP32 en Wokwi conectados.
- Misma VM con Python 3.9+, azure-iot-device y un cliente MQTT (paho-mqtt o equivalente).
- Credenciales del dispositivo Python (ID Scope, Device ID, Primary Key o SAS) en variables de entorno.
- Wokwi con el proyecto del Lab 2 funcionando.

4. Procedimiento

Etapla 1 — Abrir la caja negra del Lab 2

Sobre el dispositivo Python y el de Wokwi ya conectados a IoT Central: anotar hostname al que resuelven, puerto, si hay TLS y cada cuántos segundos publican las 3 variables. En IoT Central, abrir Data explorer y confirmar que el JSON de telemetría coincide con la plantilla.

Checkpoint: el grupo puede decir, sin adivinar, que el transporte es MQTT/TLS hacia Central.

Etapla 2 — Cliente MQTT explícito en la VM

En la VM, un segundo script (dispositivo distinto o el mismo, justificado) se conecta a IoT Central sin pasar por azure-iot-device: generar SAS, conectar con TLS, publicar las 3 variables al topic de telemetría y, si alcanza, suscribirse al topic de C2D / twin para ver un comando. El dato debe aparecer en la misma aplicación del Lab 1.

Checkpoint: un publish hecho "a mano" con MQTT se ve en IoT Central.

Etapla 3 — Mediciones sobre IoT Central

Con N mensajes de las 3 variables, registrar al menos:

- Tamaño del payload JSON (bytes).
- Intervalo configurado vs intervalo observado en Data explorer.
- Tiempo aproximado desde el publish hasta que el valor aparece en Central.
- Qué ocurre si se corta Internet 10–20 s y se restablece (SDK vs MQTT explícito vs Wokwi).
- Resultado de intentar QoS 0, QoS 1 y, si lo prueban, QoS 2 hacia Central.

Checkpoint: hay una tabla con esas columnas, no solo capturas de pantalla.

Etapla 4 — Comparación y conclusiones

Tabla final: SDK Python · MQTT explícito · Wokwi Arduino. Criterios: facilidad de implementación, control del protocolo, payload, latencia percibida, reconexión, utilidad para el escenario del Lab 1. Una conclusión explícita: cuándo usarían el SDK y cuándo un cliente MQTT desnudo contra Central.

5. Entregables

- Informe 1–2 páginas: diagrama del flujo MQTT → DPS/Hub → IoT Central, tabla de mediciones, tabla comparativa de los tres caminos y conclusión de QoS.
- Repo: script SDK (Lab 2) + script MQTT explícito + proyecto Wokwi actualizado si hubo cambios + README (cómo generar el SAS, sin secretos en claro) + /evidencias (logs de connect/publish y capturas de Central).

6. Rúbrica

Criterio	Peso
MQTT explícito publicando las 3 variables a IoT Central	30 %
Wokwi y SDK del Lab 2 siguen visibles en la misma aplicación	15 %
Calidad de la tabla de mediciones (payload, tiempo, corte, QoS)	25 %
Comparación SDK vs MQTT explícito vs Wokwi y conclusión accionable	15 %
Informe, README y evidencias	15 %

7. Recursos

- Communicate with IoT Hub using MQTT (Learn): topics, username, SAS, QoS admitido.
- Device Provisioning Service + MQTT (el primer salto de un dispositivo de IoT Central).
- paho-mqtt en la VM; azure-iot-device como baseline del Lab 2.
- Wokwi ESP32 Wi-Fi / TLS (mismo proyecto del Lab 2).